

Sentiment Analysis Cross-Domain

Confronto tra modelli di Machine Learning

Contenuti

- ❖ Introduzione
- ❖ Data preprocessing
 - Cleaning Data
 - Stopwords removal
 - Lemmatization
- ❖ Dataset Preparation
- ❖ Regressione Logistica
- ❖ Regressione Logistica con tuning TF-IDF
- ❖ Naive Bayes
- ❖ LSTM
- ❖ LSTM con embeddings pre-trained
- ❖ Transformers blocks (encoder only)
- ❖ BERT con fine-tuning
- ❖ Dataset Augmentation
- ❖ Sintesi dei Risultati
- ❖ Riferimenti

Introduzione: sentiment analysis su IMDb e Yelp

- Task di **sentiment analysis** -> classificazione binaria del sentimento espresso da frasi in **lingua inglese**.
- Frasi provenienti da due diversi domini applicativi: **IMDb** (film) e **Yelp** (attività commerciali).
- **Obiettivo**: addestrare, valutare e confrontare diversi modelli di ML, al fine di identificare il modello che presenta prestazioni migliori su ciascun dominio e su un dataset congiunto dei due.



Data preprocessing

	review	sentiment	domain
0	This place is pretty good, nice little vibe in...	1	yelp
1	I would give this television series a 10 plus ...	1	imdb
2	Everything was gross.	0	yelp
3	One of the most boring,pointless movies I have...	0	imdb
4	I hate to disagree with my fellow Yelpers, but...	0	yelp
5	Stopped by this place while in Madison for the...	1	yelp
6	This movie is great--especially if you enjoy v...	1	imdb
7	Ryan's Bar is definitely one Edinburgh establi...	0	yelp
8	One character is totally annoying with a voice...	0	imdb
9	A couple of months later, I returned and had a...	1	yelp

Dataset originale

Clean Text

	review	sentiment	domain
0	this place is pretty good nice little vibe in ...	1	yelp
1	i would give this television series a 10 plus ...	1	imdb
2	everything was gross	0	yelp
3	one of the most boring pointless movies i have...	0	imdb
4	i hate to disagree with my fellow yelpers but ...	0	yelp
5	stopped by this place while in madison for the...	1	yelp
6	this movie is great especially if you enjoy vi...	1	imdb
7	ryan s bar is definitely one edinburgh establi...	0	yelp
8	one character is totally annoying with a voice...	0	imdb
9	a couple of months later i returned and had an...	1	yelp

	review	sentiment	domain
0	place pretty good nice little vibe restaurant	1	yelp
1	would give television series 10 plus could	1	imdb
2	everything gross	0	yelp
3	one boring pointless movies ever seen	0	imdb
4	hate disagree fellow yelpers husband disappoin...	0	yelp
5	stopped place madison ironman friendly kind staff	1	yelp
6	movie great especially enjoy visual arts	1	imdb
7	ryan bar definitely one edinburgh establishmen...	0	yelp
8	one character totally annoying voice gives fee...	0	imdb
9	couple months later returned amazing meal	1	yelp

Stopwords Removal

Lemmatization

	review	sentiment	domain
0	place pretty good nice little vibe restaurant	1	yelp
1	would give television series 10 plus could	1	imdb
2	everything gross	0	yelp
3	one boring pointless movie ever see	0	imdb
4	hate disagree fellow yelpers husband disappoin...	0	yelp
5	stopped place madison ironman friendly kind staff	1	yelp
6	movie great especially enjoy visual art	1	imdb
7	ryan bar definitely one edinburgh establishmen...	0	yelp
8	one character totally annoying voice give feel...	0	imdb
9	couple month later return amazing meal	1	yelp

Dataset preparation

- Vengono considerati tre dataset:

- recensioni **IMDb**
- recensioni **Yelp**

1000 recensioni ciascuno:

- 500 positive
- 500 negative

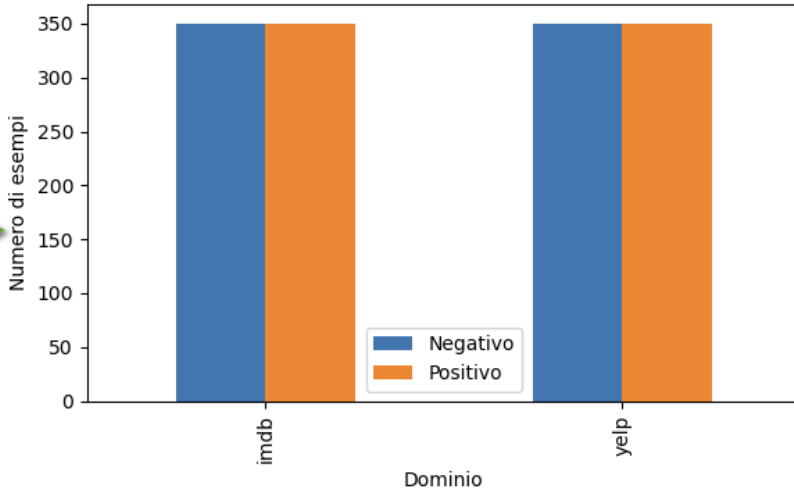
Stratificazione rispetto a classe target

- **dataset unificato** dei due domini

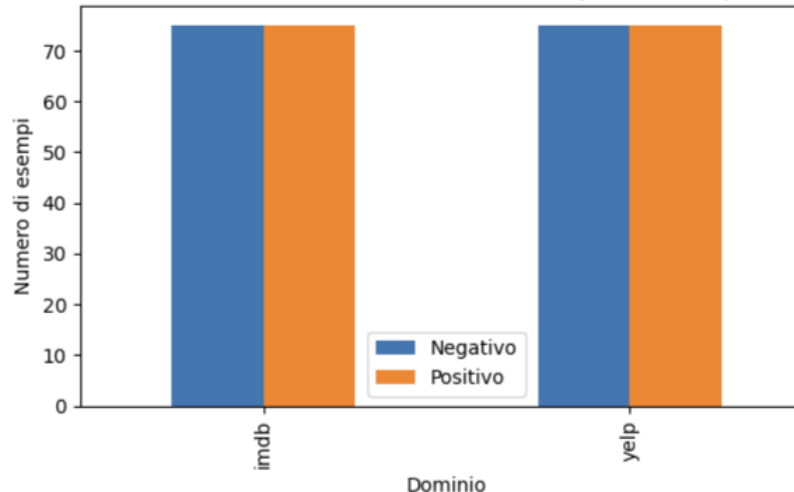
Stratificazione rispetto a classe target e al dominio

- Ogni dataset viene suddiviso in **training set** (70%), **validation set** (15%) e **test set** (15%).

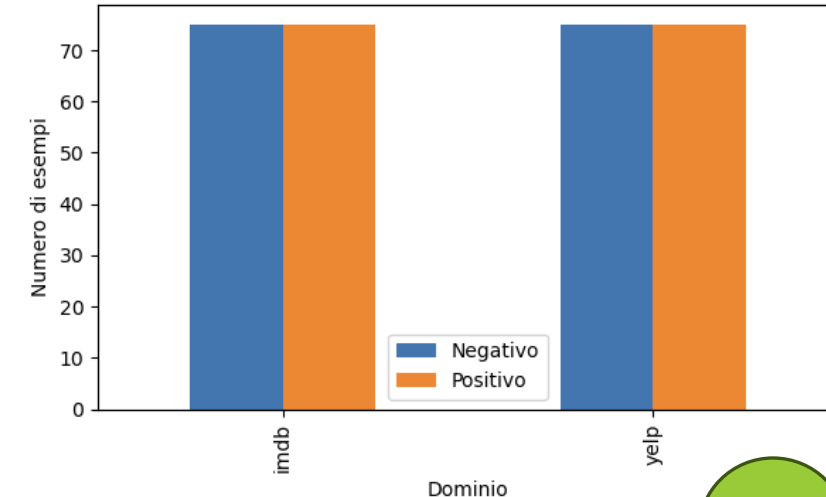
Distribuzione dominio × sentiment (TRAIN)



Distribuzione dominio × sentiment (VALIDATION)



Distribuzione dominio × sentiment (TEST)



Regressione Logistica (1/3)

Vettorizzazione TF-IDF

Ricerca iperparametri ottimali

Modello Definitivo

Evaluation

Dati testuali

Training set

fit_transform
(costruzione
vocabolario)

Training set
vettorizzato

Griglia di iperparametri
(solver, C, l1_ratio)

GridSearchCV

**parametri
ottimali**

Addestramento su
intero training set

5-fold cross validation

Modello LR
ottimizzato

Valutazione su
nuovi dati
(Accuracy e F1)

Validation set

transform
(utilizza
vocabolario)

Validation set
vettorizzato

Test set

transform
(utilizza
vocabolario)

Test set
vettorizzato

Regressione Logistica (2/3)



Parametri ottimali

`{'C': 1, 'l1_ratio': 0, 'solver': 'newton-cg'}`

`{'C': 10, 'l1_ratio': 0.5, 'solver': 'saga'}`

Metriche

IMDb VALIDATION SET
Accuracy: 0.84

Classification Report:				
	precision	recall	f1-score	support
0	0.87	0.80	0.83	75
1	0.81	0.88	0.85	75
accuracy			0.84	150
macro avg	0.84	0.84	0.84	150
weighted avg	0.84	0.84	0.84	150

IMDb TEST SET
Accuracy: 0.8266666666666667

Classification Report:				
	precision	recall	f1-score	support
0	0.83	0.83	0.83	75
1	0.83	0.83	0.83	75
accuracy			0.83	150
macro avg	0.83	0.83	0.83	150
weighted avg	0.83	0.83	0.83	150

YELP VALIDATION SET
Accuracy: 0.84

Classification Report:				
	precision	recall	f1-score	support
0	0.84	0.84	0.84	75
1	0.84	0.84	0.84	75
accuracy			0.84	150
macro avg	0.84	0.84	0.84	150
weighted avg	0.84	0.84	0.84	150

YELP TEST SET
Accuracy: 0.7533333333333333

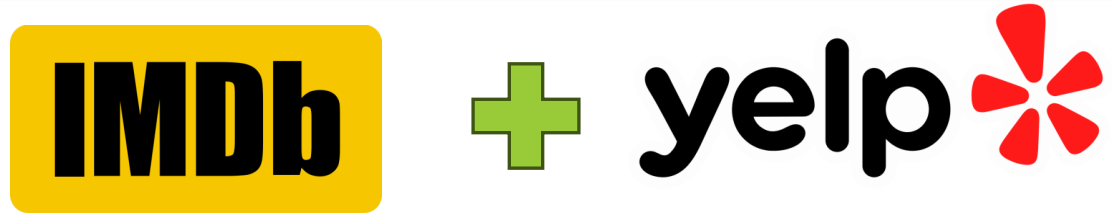
Classification Report:				
	precision	recall	f1-score	support
0	0.77	0.72	0.74	75
1	0.74	0.79	0.76	75
accuracy			0.75	150
macro avg	0.75	0.75	0.75	150
weighted avg	0.75	0.75	0.75	150

Cross-validation accuracy:
 0.764 ± 0.014

Normale differenza tra
validation e test set:

- Variabilità statistica
- Test set più complesso

Regressione Logistica (3/3)



Parametri ottimali

`{'C': 1, 'l1_ratio': 0, 'solver': 'lbfgs'}`

VALIDATION SET
Accuracy: 0.8033333333333333

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.78	0.80	150
1	0.79	0.83	0.81	150
accuracy			0.80	300
macro avg	0.80	0.80	0.80	300
weighted avg	0.80	0.80	0.80	300

TEST SET
Accuracy: 0.81

Classification Report:				
	precision	recall	f1-score	support
0	0.79	0.85	0.82	150
1	0.83	0.77	0.80	150
accuracy			0.81	300
macro avg	0.81	0.81	0.81	300
weighted avg	0.81	0.81	0.81	300

Dettaglio prestazioni per dominio:
Dominio IMDB: 121/150 corretti (80.67%)
Dominio YELP: 122/150 corretti (81.33%)

Metriche

Regressione Logistica con tuning TF-IDF (1/3)

- Si fissano i parametri ottimali della regressione logistica ottenuti in precedenza.
- Viene quindi realizzato il tuning della **vettorizzazione TF-IDF**.

```
# Definizione delle possibili configurazioni di iperparametri per tf-idf
param_grid= {
    "tfidf__analyzer": ['word', 'char', 'char_wb'],
    "tfidf__ngram_range": [(1,1), (1,2), (2,2)],
    "tfidf__min_df": [1, 2, 5],
    "tfidf__max_df": [0.9, 0.95, 1.0],
    "tfidf__binary": [True, False],
    "tfidf__use_idf": [True, False],
    "tfidf__sublinear_tf": [True, False],

    "clf__C": [0.1, 1, 10, 100]
}
```

Regressione Logistica con tuning TF-IDF (2/3)

Parametri ottimali

Metriche



'C': 1
'analyzer': 'word'
'binary': False
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 1)
'sublinear_tf': True
'use_idf': True

IMDB VALIDATION SET
Accuracy: 0.8466666666666667

Classification Report:				
	precision	recall	f1-score	support
0	0.87	0.81	0.84	75
1	0.82	0.88	0.85	75
accuracy			0.85	150
macro avg	0.85	0.85	0.85	150
weighted avg	0.85	0.85	0.85	150

IMDB TEST SET
Accuracy: 0.82

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.81	0.82	75
1	0.82	0.83	0.82	75
accuracy			0.82	150
macro avg	0.82	0.82	0.82	150
weighted avg	0.82	0.82	0.82	150



'C': 100
'analyzer': 'word'
'binary': True
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 2)
'sublinear_tf': True
'use_idf': True

YELP VALIDATION SET
Accuracy: 0.82

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.83	0.82	75
1	0.82	0.81	0.82	75
accuracy			0.82	150
macro avg	0.82	0.82	0.82	150
weighted avg	0.82	0.82	0.82	150

YELP TEST SET
Accuracy: 0.7666666666666667

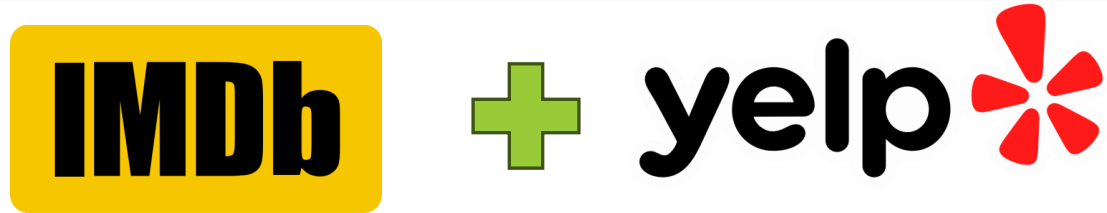
Classification Report:				
	precision	recall	f1-score	support
0	0.75	0.80	0.77	75
1	0.79	0.73	0.76	75
accuracy			0.77	150
macro avg	0.77	0.77	0.77	150
weighted avg	0.77	0.77	0.77	150

Cross-validation accuracy:
 0.771 ± 0.021

Normale differenza tra
validation e test set:

- Variabilità statistica
- Test set più complesso

Regressione Logistica con tuning TF-IDF (3/3)



Parametri ottimali

```
'C': 1
'analyzer': 'word'
'binary': False
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 1)
'sublinear_tf': True
'use_idf': True
```

Conclusioni:

- risultati simili al caso senza tuning TF-IDF
- uso congiunto di bigrammi e unigrammi su yelp
- uso di tf binario su yelp
- regolarizzazione più debole su yelp

↓ differenze su yelp

- recensioni yelp più brevi e colloquiali
- tf binario -> modello più robusto a testi brevi
- regolarizzazione più debole -> pattern più complessi

Metriche

VALIDATION SET
Accuracy: 0.8066666666666666

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.78	0.80	150
1	0.79	0.83	0.81	150
accuracy			0.81	300
macro avg	0.81	0.81	0.81	300
weighted avg	0.81	0.81	0.81	300

TEST SET
Accuracy: 0.81

Classification Report:				
	precision	recall	f1-score	support
0	0.79	0.85	0.82	150
1	0.84	0.77	0.80	150
accuracy			0.81	300
macro avg	0.81	0.81	0.81	300
weighted avg	0.81	0.81	0.81	300

Dettaglio prestazioni per dominio:

Dominio IMDB: 120/150 corretti (80.00%)
Dominio YELP: 123/150 corretti (82.00%)

Naive Bayes (1/3)

- Modello probabilistico generativo.
- Anche in questo caso input fornito sotto forma di singolo vettore.
- Tuning congiunto sia del modello che del tuning TF-IDF.

```
param_grid= {  
    "tfidf__analyzer": ['word', 'char', 'char_wb'],  
    "tfidf__ngram_range": [(1,1), (1,2), (2,2)],  
    "tfidf__min_df": [1, 2, 5],  
    "tfidf__max_df": [0.9, 0.95, 1.0],  
    "tfidf__binary": [True, False],  
    "tfidf__use_idf": [True, False],  
    "tfidf__sublinear_tf": [True, False],  
  
    "mnbc__alpha": [0.001, 0.01, 0.1, 0.5, 1.0],  
    "mnbc__fit_prior": [True, False]  
}
```

Iperparametri modello:

- alpha = fattore di smoothing di Laplace
- fit_prior = probabilità a priori apprese o uniformi

Naive Bayes (2/3)

Parametri ottimali

Metriche

IMDb

'alpha': 1.0
'fit_prior': True
'analyzer': 'word'
'binary': True
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 2)
'sublinear_tf': True
'use_idf': False

IMDb VALIDATION SET
Accuracy: 0.8466666666666667

Classification Report:				
	precision	recall	f1-score	support
0	0.83	0.87	0.85	75
1	0.86	0.83	0.84	75
accuracy			0.85	150
macro avg	0.85	0.85	0.85	150
weighted avg	0.85	0.85	0.85	150

IMDb TEST SET
Accuracy: 0.8

Classification Report:				
	precision	recall	f1-score	support
0	0.81	0.79	0.80	75
1	0.79	0.81	0.80	75
accuracy			0.80	150
macro avg	0.80	0.80	0.80	150
weighted avg	0.80	0.80	0.80	150

yelp

'alpha': 0.1
'fit_prior': True
'analyzer': 'word'
'binary': False
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 1)
'sublinear_tf': True
'use_idf': False

YELP VALIDATION SET
Accuracy: 0.82

Classification Report:				
	precision	recall	f1-score	support
0	0.84	0.79	0.81	75
1	0.80	0.85	0.83	75
accuracy			0.82	150
macro avg	0.82	0.82	0.82	150
weighted avg	0.82	0.82	0.82	150

YELP TEST SET
Accuracy: 0.76

Classification Report:				
	precision	recall	f1-score	support
0	0.75	0.79	0.77	75
1	0.77	0.73	0.75	75
accuracy			0.76	150
macro avg	0.76	0.76	0.76	150
weighted avg	0.76	0.76	0.76	150

Cross-validation accuracy:
 0.773 ± 0.038

Normale differenza tra
validation e test set:

- Variabilità statistica
- Test set più complesso

Naive Bayes (3/3)



Parametri ottimali

```
'alpha': 0.1
'fit_prior': True
'analyzer': 'word'
'binary': False
'max_df': 0.9
'min_df': 1
'ngram_range': (1, 2)
'sublinear_tf': True
'use_idf': False
```

VALIDATION SET
Accuracy: 0.82

Classification Report:				
	precision	recall	f1-score	support
0	0.82	0.81	0.82	150
1	0.82	0.83	0.82	150
accuracy			0.82	300
macro avg	0.82	0.82	0.82	300
weighted avg	0.82	0.82	0.82	300

TEST SET
Accuracy: 0.8

Classification Report:				
	precision	recall	f1-score	support
0	0.78	0.83	0.81	150
1	0.82	0.77	0.79	150
accuracy			0.80	300
macro avg	0.80	0.80	0.80	300
weighted avg	0.80	0.80	0.80	300

Conclusioni:

- risultati simili alla regressione logistica
- uso congiunto di unigrammi e bigrammi su imdb e dataset unificato
- non utilizza termine di idf

↓ motivazioni

- uso unigrammi e bigrammi -> assunzione feature indipendenti -> vantaggio più feature
- Usa solo termine tf -> stima $P(x|c)$ dai dati -> vantaggio conteggio raw

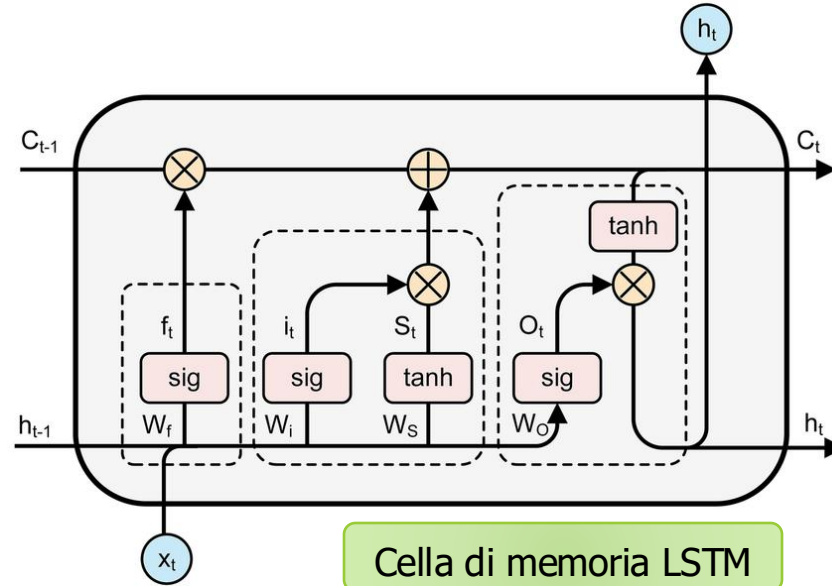
Dettaglio prestazioni per dominio:

Dominio IMDB: 117/150 corretti (78.00%)
Dominio YELP: 123/150 corretti (82.00%)

Metriche

LSTM (1/5)

- Queste reti, così come le successive, considerano sequenze di token in input piuttosto che singoli vettori.
- Reti LSTM preferite a reti tradizionali perché fanno uso di celle di memoria, che permettono di mitigare il problema del vanishing/exploding gradients e la perdita di bit di informazione.
- Si considera, così come per i modelli successivi, input testuale non pre-processato (solo clean text).



LSTM (2/5)

Input

pre-processamento

Addestramento del modello

Evaluation

Dati testuali
grezzi

tokenizer

fit_on_texts

costruzione
vocabolario
(train set)

text_to_sequences

testo come
sequenze
di indici

si considera
train set

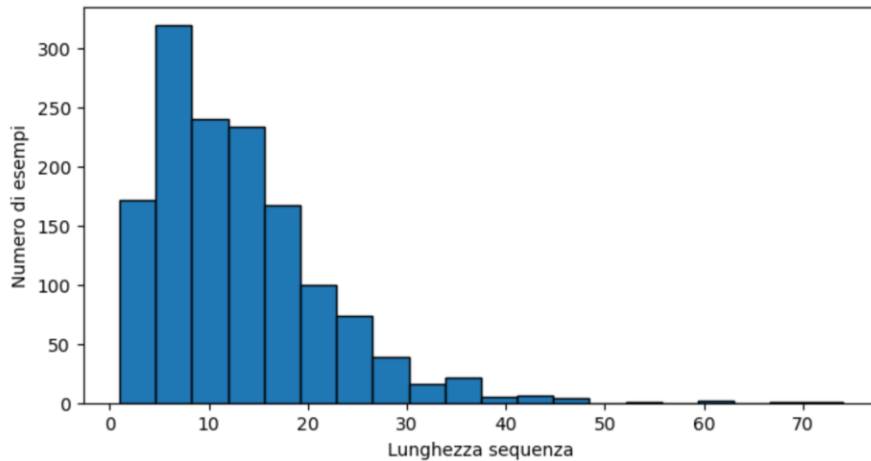
Training rete LSTM

Early stopping con
validation set

Valutazione su test set

Raccolta metriche
(Accuracy e F1)

Dataset unificato



pad_sequences

padding/
troncamento
(MAX_LEN=30)

Esempio tokenizzazione:

my first visit to hiro was a delight → [34, 125, 649, 9, 1382, 8, 4, 865]

LSTM (3/5)

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 30, 128)	446,976
lstm (LSTM)	(None, 64)	49,408
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dropout_1 (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 1)	33

EMBED_DIM = 128 -> trade-off tra capacità espressiva e overfitting

Rete non troppo deep -> evitare rischio di overfitting e training instabile

Dropout -> mitiga l'overfitting. Miglioramento delle prestazioni

Total params: 498,497 (1.90 MB)

Trainable params: 498,497 (1.90 MB)

Anche gli embeddings sono appresi durante il training

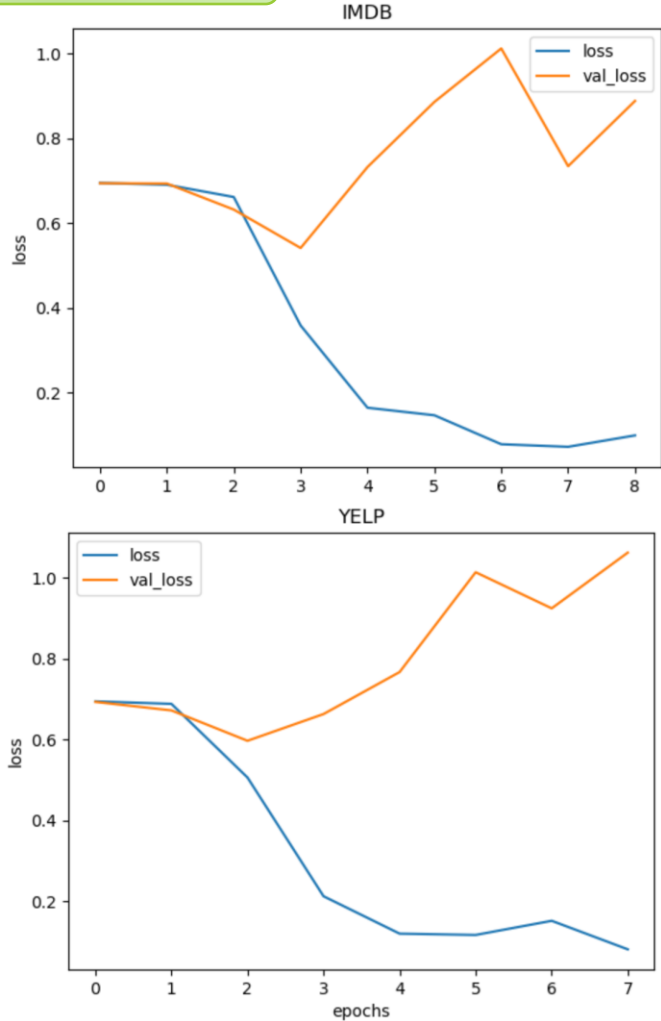
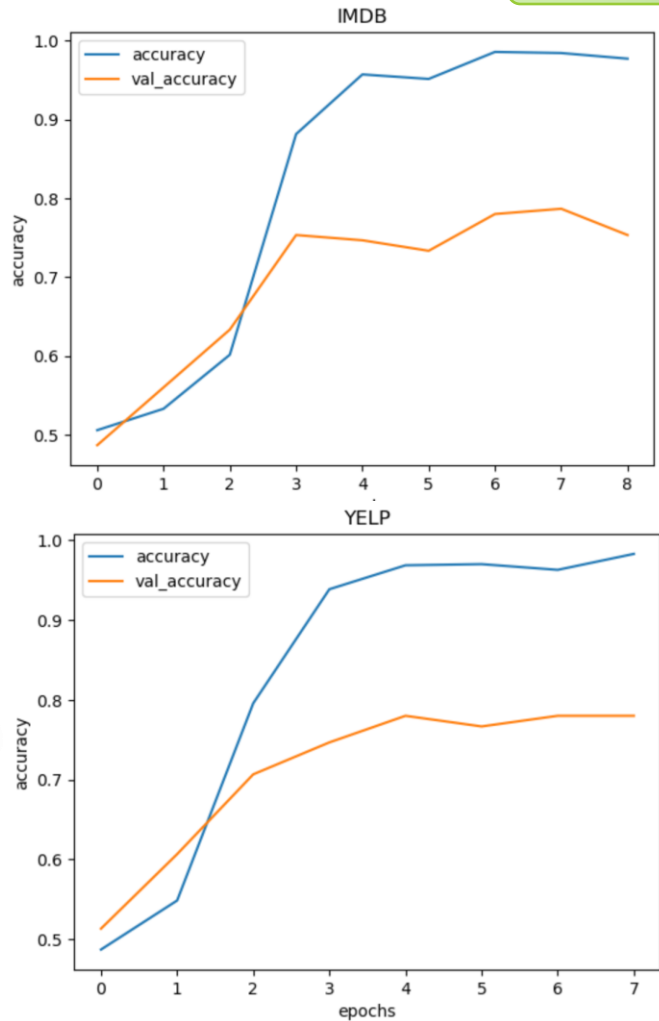
Non-trainable params: 0 (0.00 B)

LSTM (4/5)

IMDb

yelp 

Andamento training



Metriche

Accuracy: 0.827
F1 score: 0.827

Accuracy: 0.767
F1 score: 0.766

LSTM (5/5)

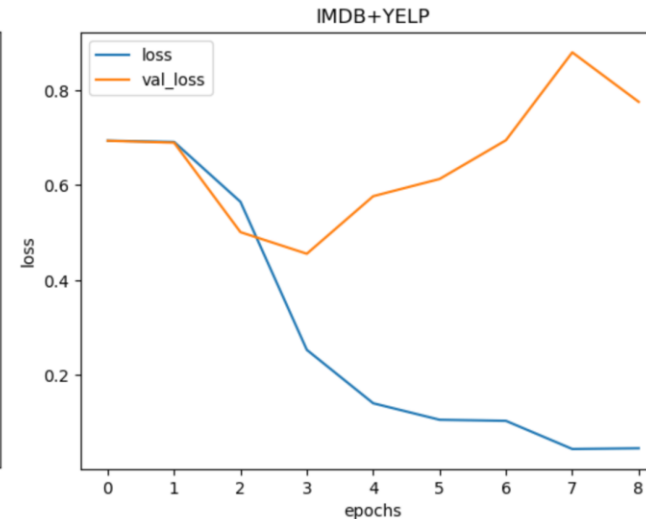
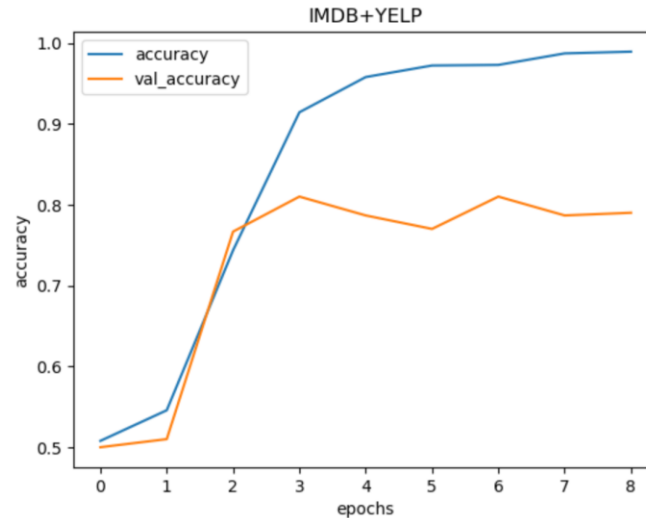
Andamento training

Metriche

IMDb



yelp



Accuracy: 0.770
F1 score: 0.768

Conclusioni:

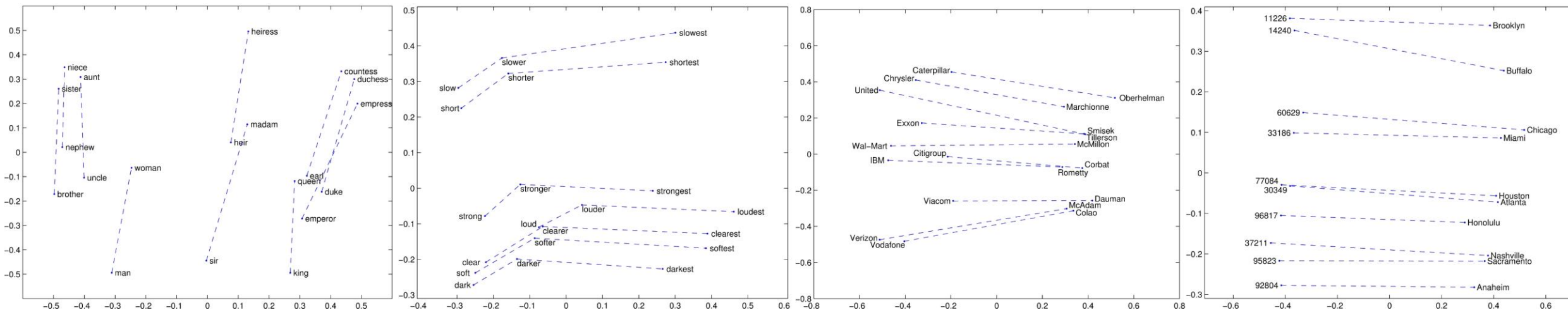
- In tutti e tre i casi risultati simili (o peggiori) alla baseline
- Pochi dati -> rete va rapidamente in overfitting
- Non riesce ad apprendere embeddings significativi

Dettaglio prestazioni per dominio:

Dominio IMDb: 109/150 corretti (72.67%)
Dominio YELP: 122/150 corretti (81.33%)

LSTM con pretrained embeddings (1/4)

- Strategia per mitigare il problema dei pochi dati a disposizione e ridurre il numero di parametri della rete -> **utilizzo di embeddings GloVe**.
- Parole rappresentate in uno spazio vettoriale in cui termini semanticamente simili risultano geometricamente vicini.
- Dimensione degli embeddings = 300 -> tramite diversi tentativi è risultato che la rete trae maggiore vantaggio da una **dimensionalità maggiore degli embeddings pre-trained**.



Embeddings GloVe

LSTM con pretrained embeddings (2/4)

```
def build_LSTM_pretrained(vocab_size, embeddings):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(MAX_LEN,)),
        tf.keras.layers.Embedding(vocab_size, 300, weights = [embeddings], input_length=MAX_LEN, trainable = False),
        tf.keras.layers.LSTM(64),
        tf.keras.layers.Dense(32, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

    model.compile(
        loss='binary_crossentropy',
        optimizer='adam',
        metrics=['accuracy']
    )

    return model
```

Matrice di embeddings pretrained

Rete analoga alla precedente, ma senza livelli di dropout -> non c'è rischio di overfitting sugli embeddings e numero ridotto di parametri addestrabili

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	(None, 30, 300)	747,000
lstm_8 (LSTM)	(None, 64)	93,440
dense_16 (Dense)	(None, 32)	2,080
dense_17 (Dense)	(None, 1)	33

Total params: 842,553 (3.21 MB)

Riduzione del numero di parametri addestrabili

Trainable params: 95,553 (373.25 KB)

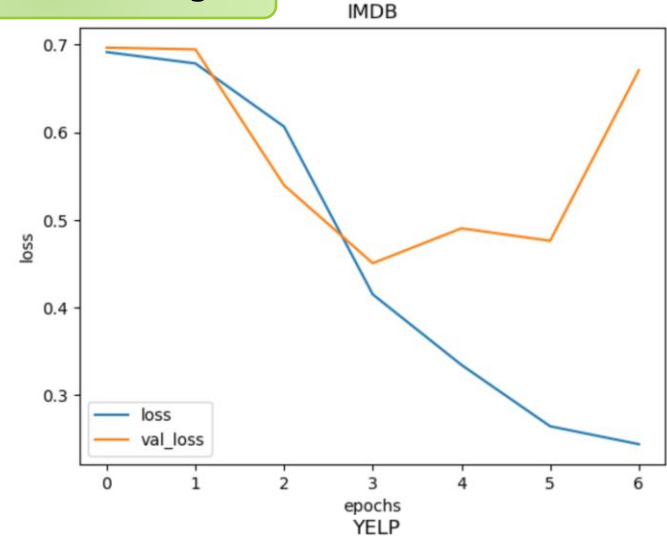
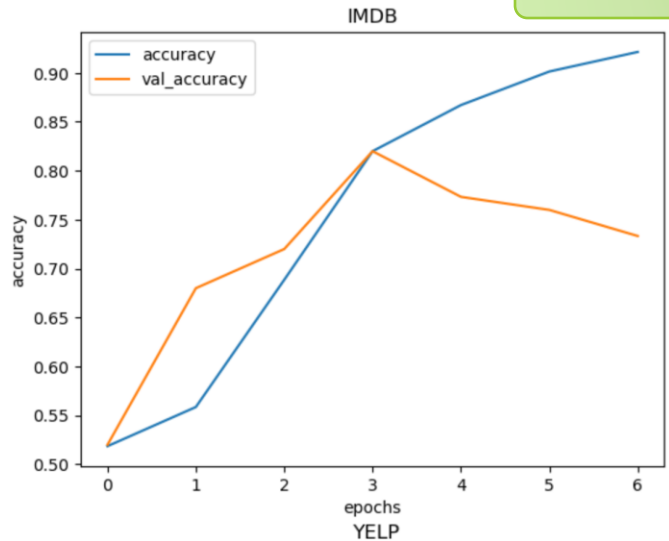
Non-trainable params: 747,000 (2.85 MB)

LSTM con pretrained embeddings (3/4)

IMDb

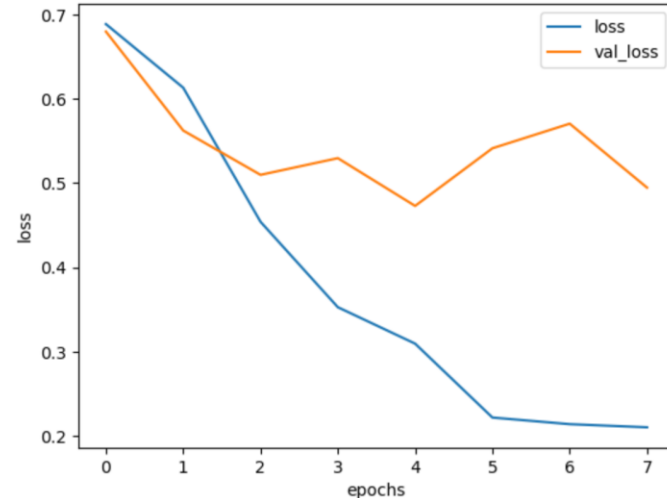
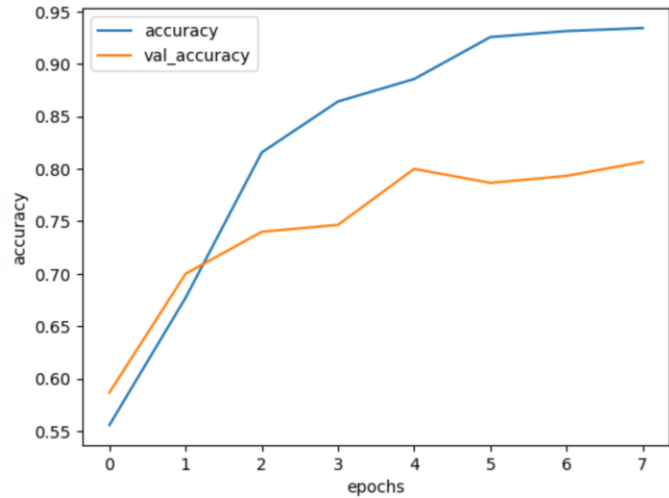
yelp 

Andamento training



Metriche

Accuracy: 0.853
F1 score: 0.853



Accuracy: 0.807
F1 score: 0.807

LSTM con pretrained embeddings (4/4)

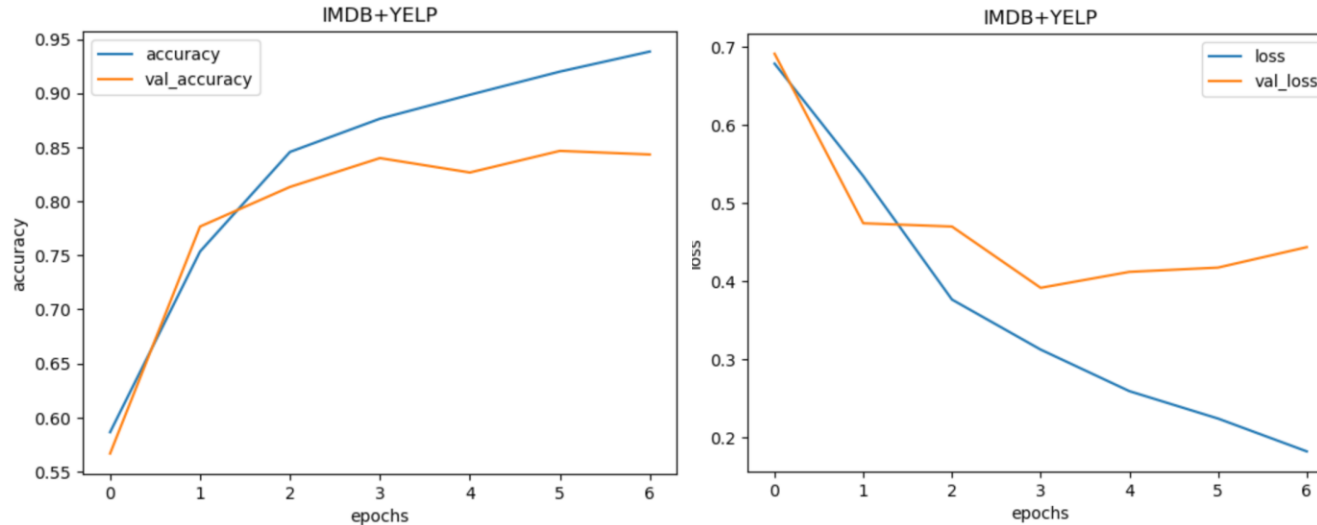
Andamento training

Metriche

IMDb



yelp



Accuracy: 0.833
F1 score: 0.833

Conclusioni:

- Prestazioni migliori rispetto a LSTM con embeddings appresi
- Prestazioni migliori rispetto a baseline sul dataset unificato -> riesce a catturare la dinamica di sequenze di token e il loro ordine
- Uso embeddings GloVe -> la rete parte da una rappresentazione semantica già significativa

Dettaglio prestazioni per dominio:

Dominio IMDb: 124/150 corretti (82.67%)
Dominio YELP: 126/150 corretti (84.00%)

Transformer encoder-only (1/4)

- Si presenta un altro modello adatto al **processamento di dati sequenziali**.
- A differenza della rete ricorrente, il transformer riceve l'intera **sequenza di token in parallelo** -> riduce tempo di training su hardware parallelo.
- Produce **embeddings contestuali** tramite meccanismo dell'attention.

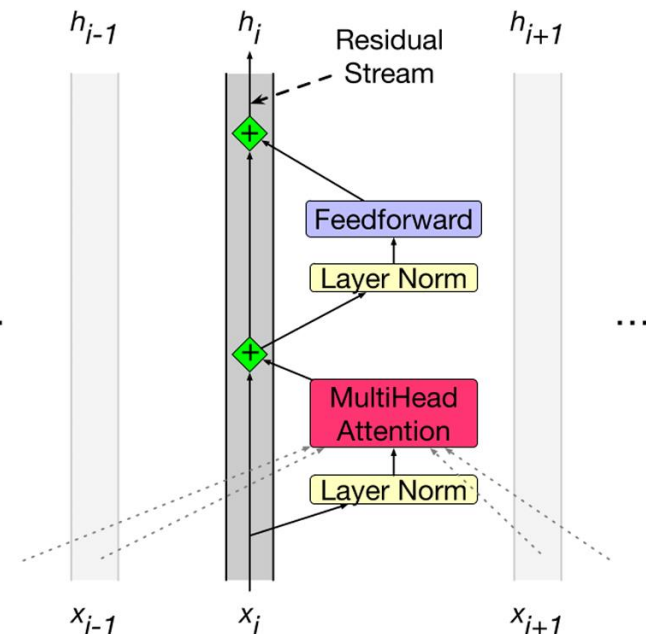
```
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads,
                                              key_dim=embed_dim)

        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])

        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training=False):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)

        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)
```



Transformer encoder-only (2/4)

Layer (type)	Output Shape	Param #
input_layer_29 (InputLayer)	(None , 30)	0
embedding_21 (Embedding)	(None , 30, 300)	747,000
add_4 (Add)	(None , 30, 300)	0
transformer_block_12 (TransformerBlock)	(None , 30, 300)	1,522,328
transformer_block_13 (TransformerBlock)	(None , 30, 300)	1,522,328
transformer_block_14 (TransformerBlock)	(None , 30, 300)	1,522,328
global_average_pooling1d_4 (GlobalAveragePooling1D)	(None , 300)	0
dense_64 (Dense)	(None , 32)	9,632
dense_65 (Dense)	(None , 1)	33

Total params: 5,323,649 (20.31 MB)

Trainable params: 5,323,649 (20.31 MB)

Non-trainable params: 0 (0.00 B)

Adattamento al task specifico

Matrici di embeddings pretrained ma addestrabili

Embeddings posizionali appresi concatenati a token embeddings

Catena di blocchi transformers -> costruiscono embeddings contestuali sempre più ottimali rispetto al task considerato -> numero blocchi = trade-off tra capacità di estrazione del contesto e numero di parametri

4 attention heads per ogni blocco -> catturano diverse relazioni contestuali tra i token in input

Rete feedforward con 128 unità per ogni blocco -> capacità espressiva sufficiente senza rischio di overfitting

Head del classificatore che estrae il sentimento espresso

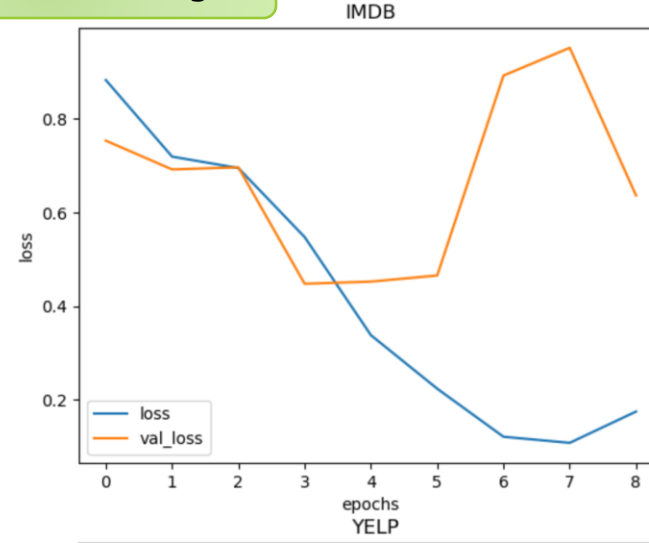
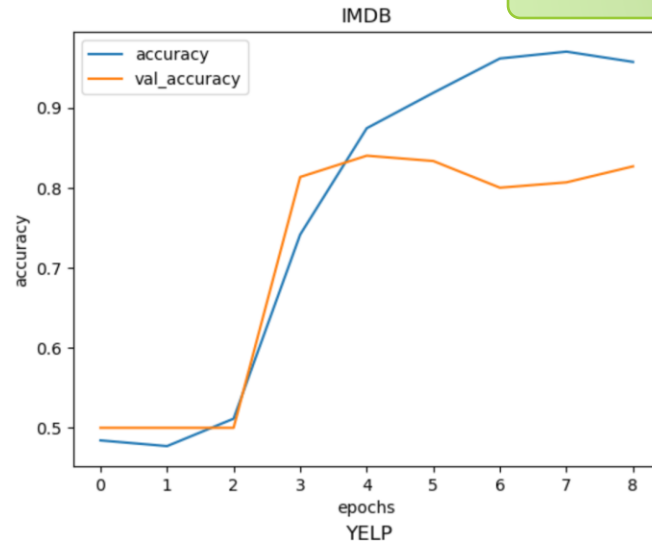
Costruisce a partire da una sequenza di embeddings contestuali, un unico vettore che rappresenta intera frase

Transformer encoder-only (3/4)

IMDb

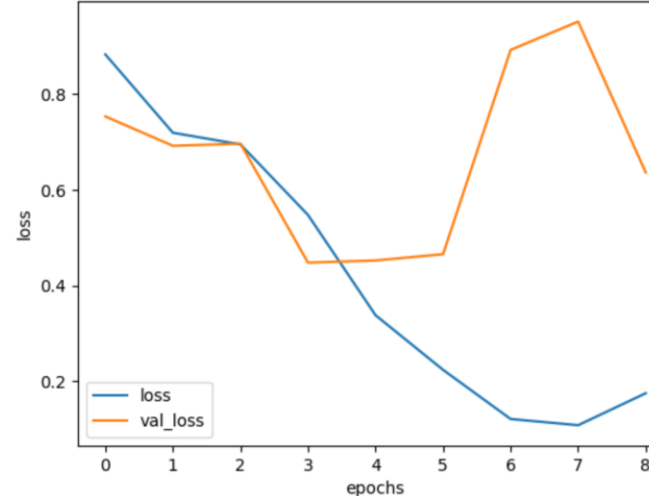
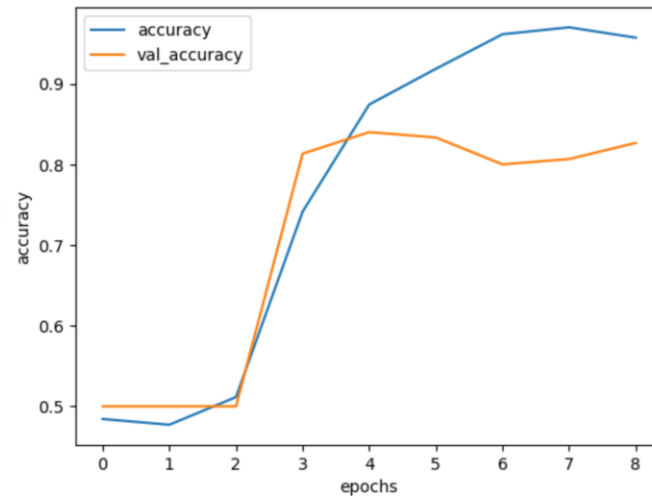
yelp 

Andamento training



Mettriche

Accuracy: 0.833
F1 score: 0.833



Accuracy: 0.820
F1 score: 0.818

Transformer encoder-only (4/4)

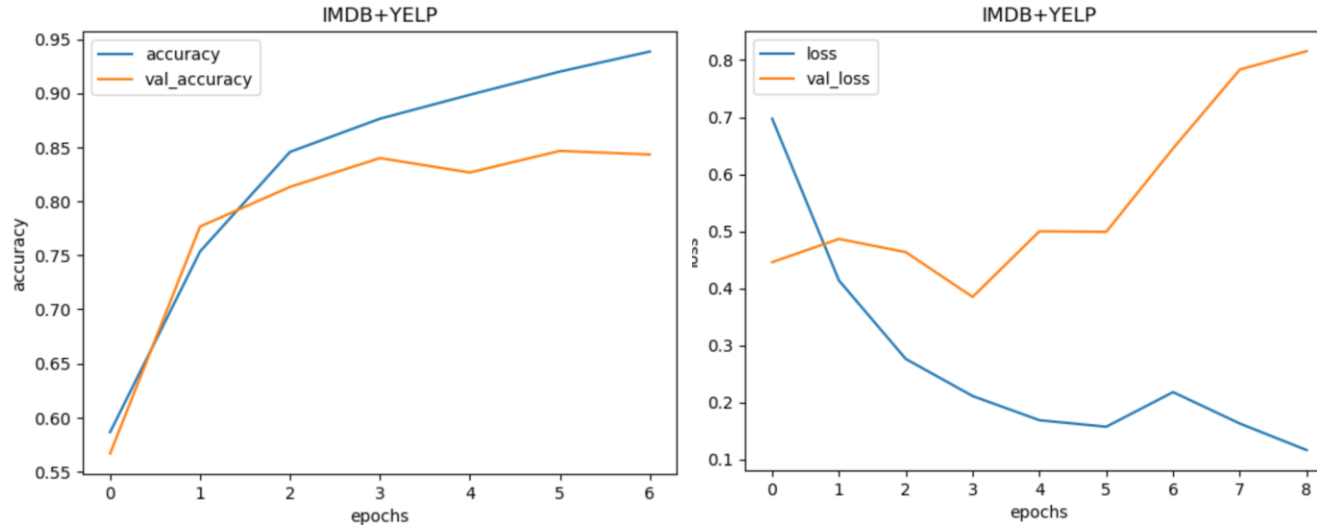
Andamento training

Metriche

IMDb



yelp



Accuracy: 0.840
F1 score: 0.840

Conclusioni:

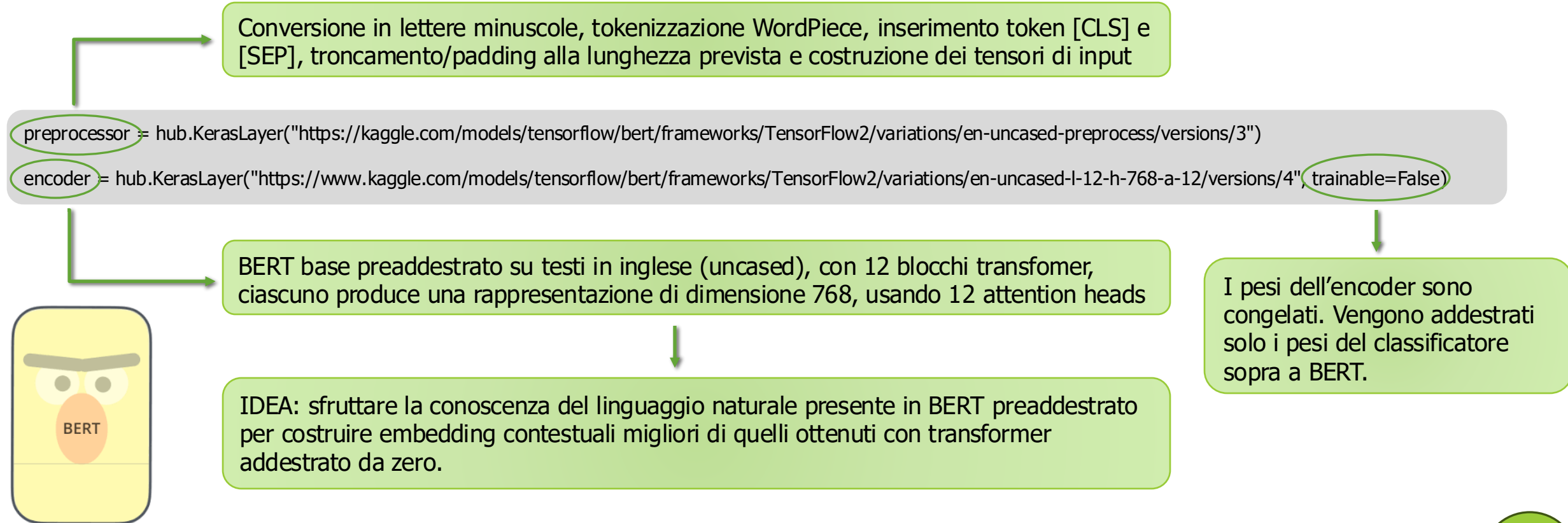
- Prestazioni di poco migliori rispetto a LSTM con pretrained embeddings (su yelp e dataset unificato)
- Maggiori vantaggi dall'uso di dataset più grande e cross-domain
- Embeddings contestuali catturano relazioni semantiche che embeddings statici non possono apprendere
- Non c'è perdita di informazione grazie al processamento in parallelo

Dettaglio prestazioni per dominio:

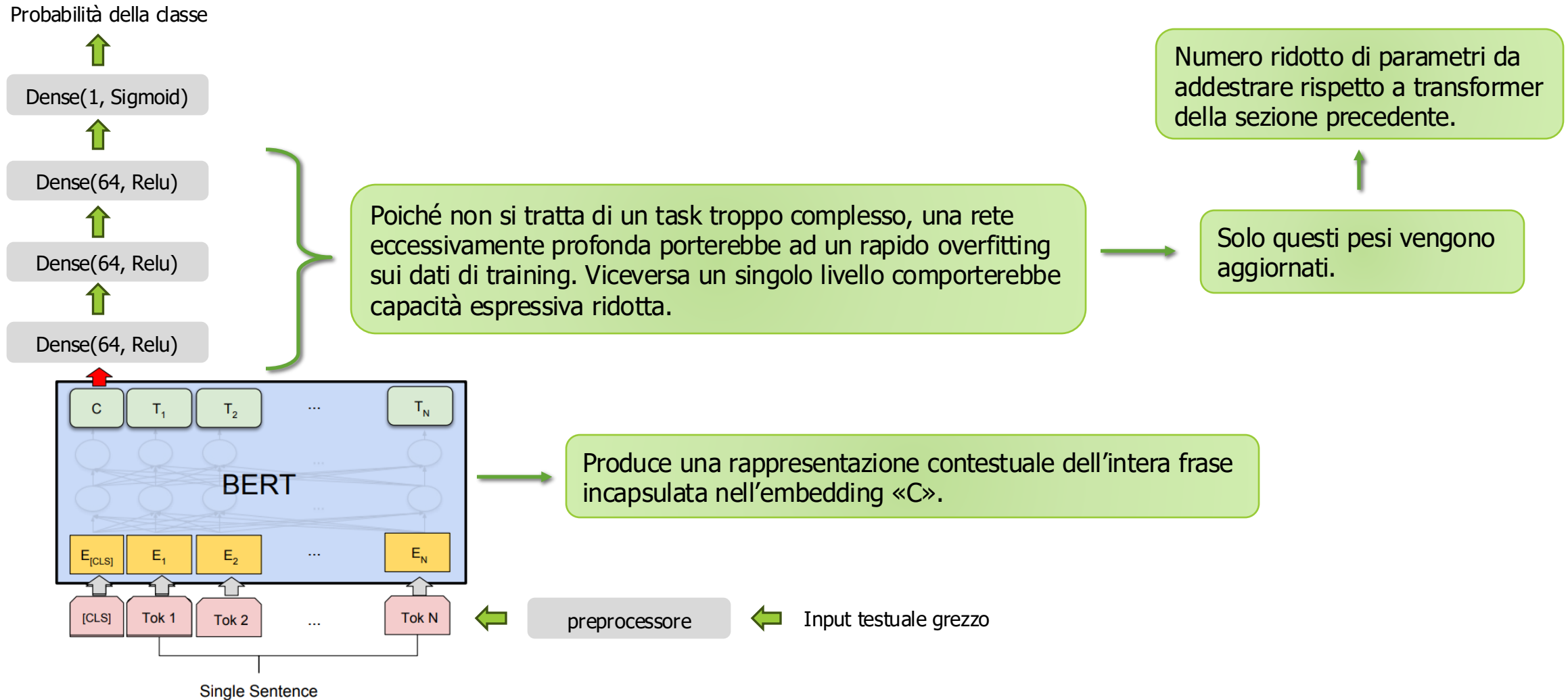
Dominio IMDb: 122/150 corretti (81.33%)
Dominio YELP: 130/150 corretti (86.67%)

BERT con fine-tuning (1/4)

- Una strategia per risolvere il problema dei pochi dati a disposizione è di utilizzare un modello pre-addestrato su cui realizzare il fine-tuning sul task considerato. Questo riduce notevolmente il numero di parametri da addestrare.



BERT con fine-tuning (2/4)

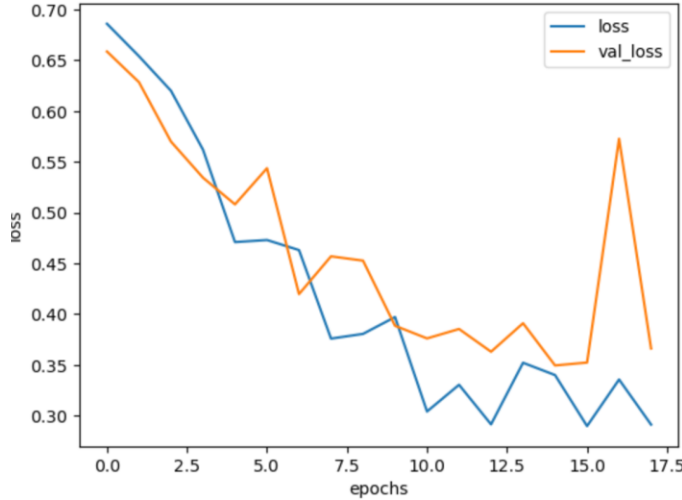
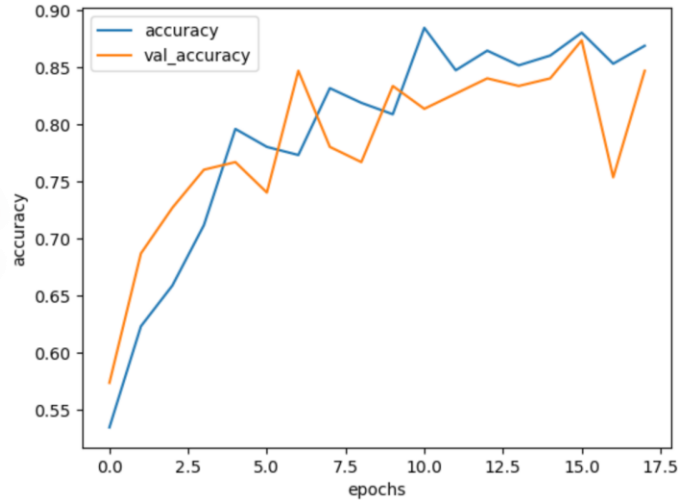
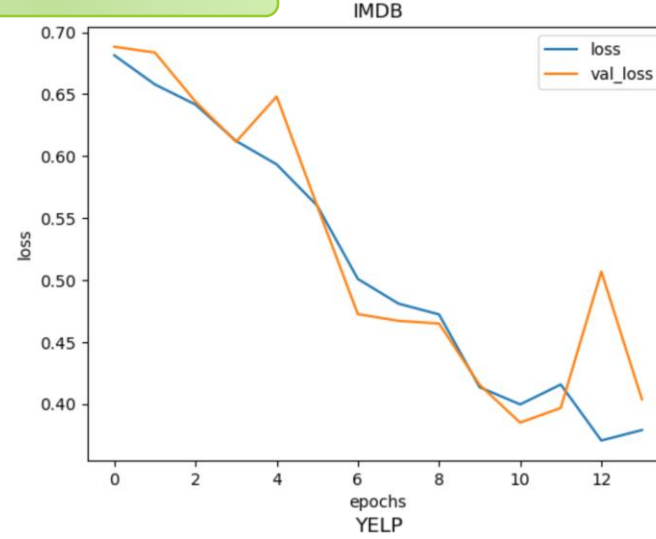
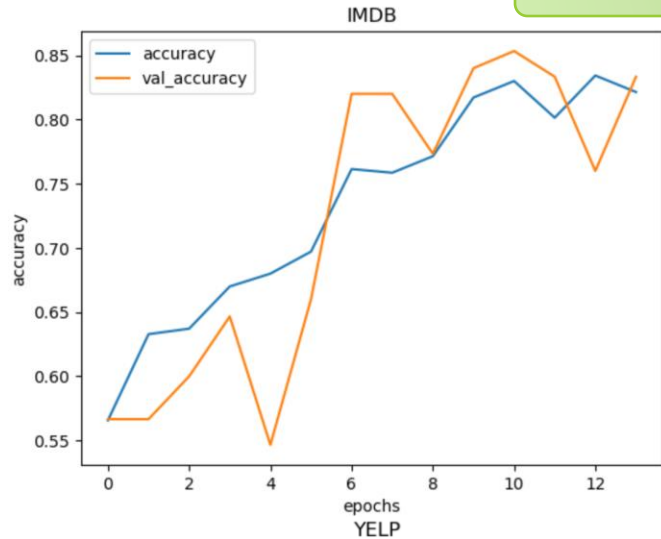


BERT con fine-tuning (3/4)

IMDb

yelp 

Andamento training



Metriche

Accuracy: 0.900
F1 score: 0.900

Accuracy: 0.833
F1 score: 0.833

BERT con fine-tuning (4/4)

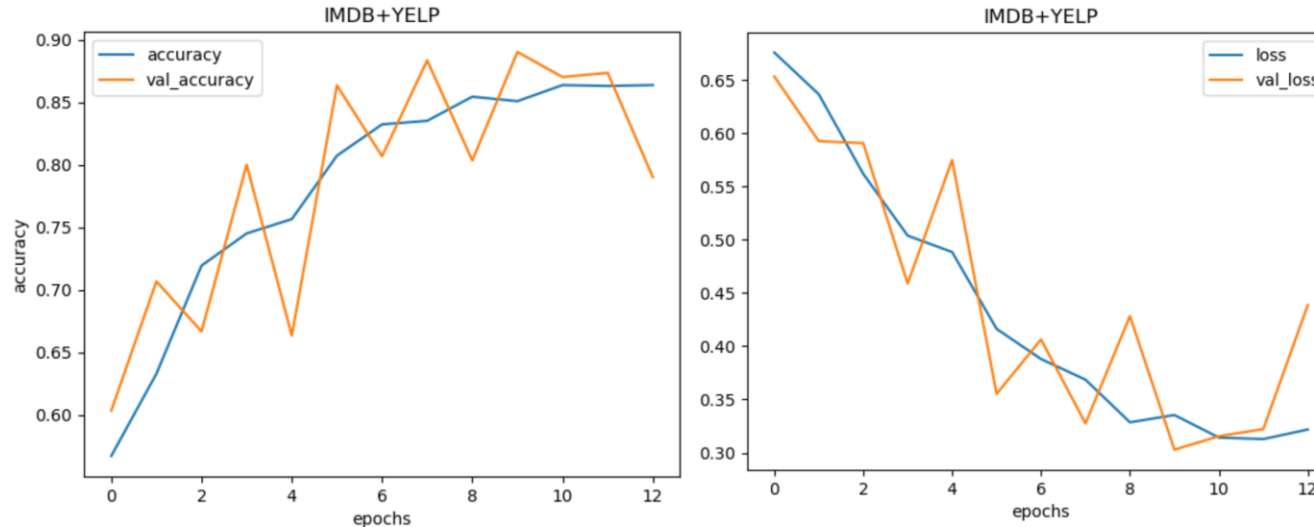
Andamento training

Metriche

IMDb



yelp



Accuracy: 0.870
F1 score: 0.870

Conclusioni:

- BERT con fine-tuning risulta il modello migliore, su entrambi i domini e sul dataset unificato.
- Pesi dell'encoder congelati -> costo computazionale a training time contenuto.
- Ottimi risultati derivano da conoscenza pregressa del linguaggio.
- Supera il problema del dataset di dimensione ridotta.
- Riesce a evitare l'overfitting, come mostrato dalle curve di training.

Dettaglio prestazioni per dominio:

Dominio IMDb: 124/150 corretti (82.67%)
Dominio YELP: 137/150 corretti (91.33%)

Dataset augmentation (1/2)

- Si tratta di un'altra strategia per cercare di **risolvere il problema dei pochi dati di training** a disposizione. Si considera il modello migliore ottenuto finora, ossia BERT con fine-tuning.
- **Obiettivo:** migliorare la capacità di generalizzazione del modello.
- Idea: **creare sinteticamente nuove frasi artificiali** a partire da quelle presenti nel training set originale.
- Si considera solo il dataset unificato, che massimizza il numero di dati a disposizione. Validation e test set vengono mantenuti invariati.



Dataset augmentation (2/2)

Consideriamo il modello migliore ottenuto, ossia **BERT con fine-tuning**.

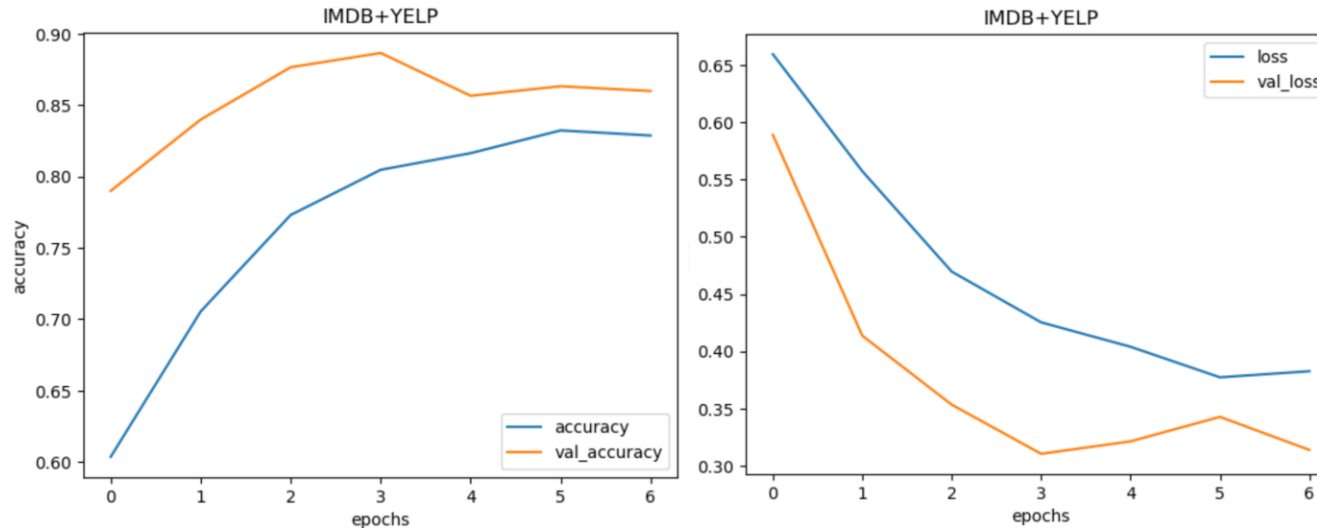
Andamento training

Metriche

IMDb



yelp

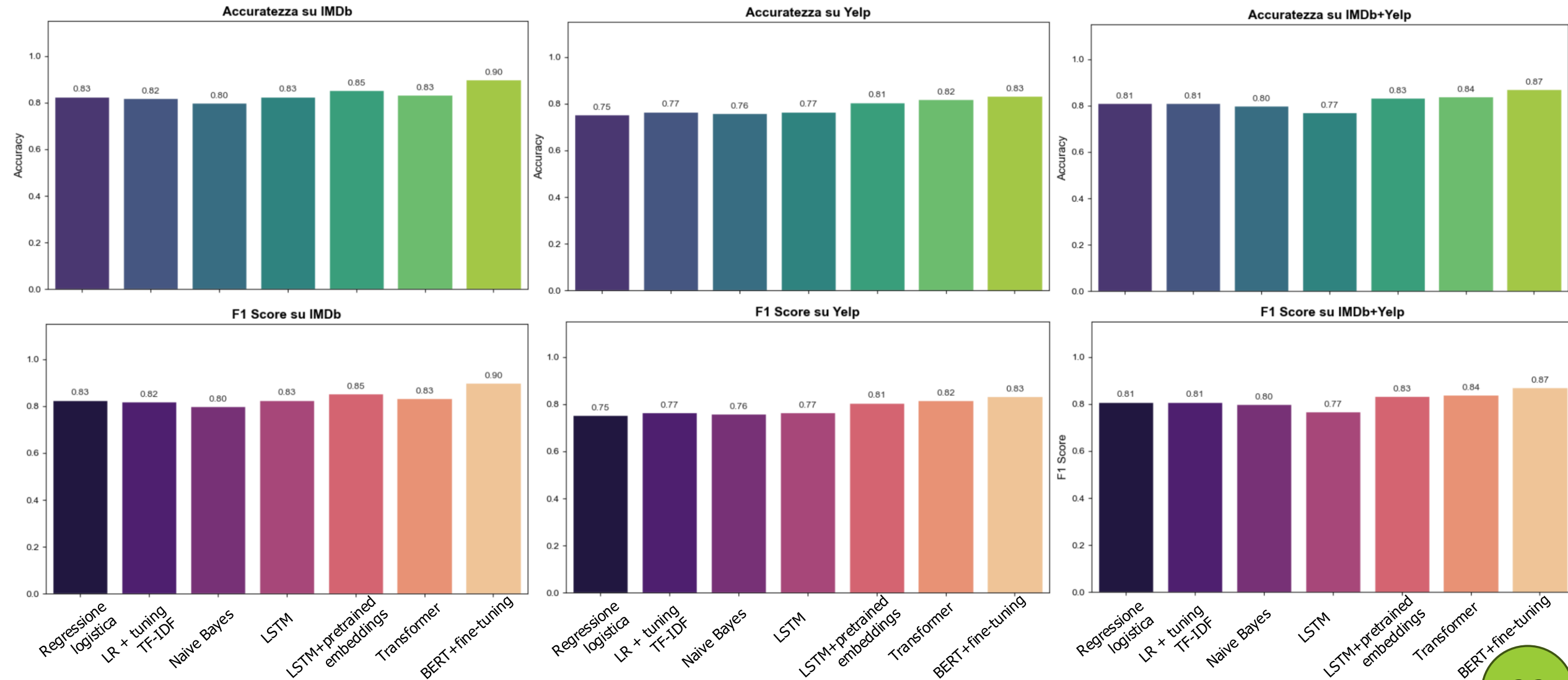


Accuracy: 0.877
F1 score: 0.877

Conclusioni:

- Non si ottengono risultati drasticamente migliori relativamente all'accuratezza ottenuta -> augmentation può essere ridondante dal punto di vista lessicale.
- Si potrebbero osservare un reale miglioramento se si avessero a disposizione nuove frasi reali, non artificiali.
- Vantaggio -> maggiore velocità di convergenza alla soluzione ottimale e curve di training più regolari.

Sintesi dei Risultati



Riferimenti

1. https://scikit-learn.org/stable/user_guide.html
2. <https://www.tensorflow.org/guide?hl=it>
3. https://keras.io/examples/nlp/text_classification_with_transformer/
4. https://www.tensorflow.org/text/tutorials/classify_text_with_bert?hl=it
5. <https://neptune.ai/blog/data-augmentation-nlp>